Especificación de Requerimientos – Colitas felices

Iteración: 2

Alumnos: Julieta Melina Flores Morán (321088642) - José Antonio Zarco Romero (321102502)- Luis Ángel Casique Abraham (321266536) - Yolanda Ximena Sánchez Cervantes (319263886)

Profesora: Karla Socorro García Alcántara

Versión del Documento: 2.0

Fecha: 10 de abril de 2026

1. Introducción

1.1 Propósito del documento

Este documento describe la evolución de los requerimientos funcionales y no funcionales del sistema Colitas Felices en el contexto de la Iteración 2 del proyecto. Su propósito es reflejar el estado actual del sistema implementado, funcionando como una referencia técnica y funcional tanto para el equipo de desarrollo como para stakeholders.

A diferencia de la iteración anterior, esta versión documenta un sistema completo que integra tanto el back-end como una interfaz de usuario funcional, permitiendo la interacción directa del usuario con la plataforma, constituyendo la primera versión funcional estable del back-end y front-end.

Asimismo, este documento sirve como base de conocimiento para futuras iteraciones, permitiendo mantener coherencia entre el diseño del sistema y su implementación real.

1.2 Alcance del sistema

El sistema Colitas Felices es una aplicación web orientada a facilitar la adopción responsable de perros y gatos en el territorio mexicano, sirviendo como punto de contacto entre personas interesadas en adoptar y entidades cuidadoras.

En la Iteración 2, el sistema evoluciona de un back-end aislado a un sistema completo compuesto por front-end, back-end y base de datos, funcionando de forma integrada.

Las funcionalidades actualmente implementadas corresponden a:

1. Registro de usuarios
2. Inicio de sesión
3. Cierre de sesión
4. Consulta de información del usuario autenticado
5. Actualización de información del usuario autenticado

Estas funcionalidades constituyen la base sobre la cual se desarrollarán futuras iteraciones del sistema.

Fuera del alcance en la Iteración 2 del sistema

Las siguientes funcionalidades, aunque contempladas para el producto final, no forman parte del sistema actualmente implementado:

- Gestión de animales en adopción
- Publicación, edición y eliminación de fichas de animales
- Visualización de animales mediante mapa o lista
- Manifestación de interés en animales
- Interacción entre usuarios
- Funcionalidades administrativas
- Integración con APIs externas
- Almacenamiento de imágenes de perfil en el servidor

Estas características serán consideradas en iteraciones posteriores del proyecto.

1.3 Definiciones relevantes

1. **Usuario**: Persona registrada que accede al sistema mediante credenciales válidas.
2. **Usuario nuevo:** Persona que no cuenta con una cuenta registrada y desea crear un perfil dentro del sistema.

3. I**nformación personal:** Datos asociados a un usuario que permiten su identificación dentro del sistema, tales como nombre, correo electrónico y otros atributos definidos en el modelo de datos.
4. **Rol de usuario:** Atributo asociado a cada usuario que indica su tipo dentro del sistema (por ejemplo, cuidador o interesado). En la Iteración 1, este atributo es almacenado, pero no define comportamientos diferenciados dentro del sistema.
5. **Autenticación**: Proceso mediante el cual el sistema verifica la identidad de un usuario a partir de sus credenciales.
6. **Usuario autenticado:** Usuario que ha iniciado sesión correctamente y cuenta con acceso a funcionalidades protegidas del sistema.

2. Actores del sistema

1. **Actor**: Usuario

   Tipo: Interno

   Descripción: Persona que interactúa con el sistema mediante la creación de una cuenta y el uso de credenciales para autenticarse. Este actor representa a cualquier individuo que accede a la plataforma en la Iteración 2.

   Responsabilidades dentro del sistema:

   - Registrarse en la plataforma proporcionando la información requerida
   - Iniciar sesión mediante sus credenciales
   - Cerrar sesión
   - Consultar su información personal
   - Actualizar su información personal

En iteraciones futuras se contempla la diferenciación de actores con base en el rol de usuario, lo cual permitirá definir comportamientos específicos como publicación de animales o manifestación de interés en adopciones.

3. Requerimientos Funcionales

Los siguientes requerimientos funcionales corresponden a las funcionalidades implementadas en la Iteración 2 del sistema, en esta cubrimos la gestión completa de usuarios desde la interfaz de usuario, incluyendo el nuevo requerimiento de actualización de información del usuario autenticado que se incorpora en esta versión.

Requerimiento		Descripción	Criterios de aceptación
Gestión de usuarios			
RF1	**Registro de usuarios**	El sistema deberá permitir a nuevos usuarios registrarse en la plataforma mediante el ingreso de su información personal, con el fin de crear una cuenta única.	El sistema deberá solicitar la información necesaria para el registro del usuario. El sistema deberá validar que el correo electrónico y el nombre de usuario no estén previamente registrados. El sistema deberá validar que la información obligatoria esté completa. El sistema deberá registrar al usuario en la plataforma si la información es válida
RF2	**Inicio de sesión**	El sistema deberá permitir a los usuarios registrados autenticarse para acceder a la plataforma.	El sistema deberá solicitar credenciales de acceso al usuario. El sistema deberá validar que las credenciales correspondan a un usuario registrado. El sistema deberá permitir el acceso únicamente cuando las credenciales sean correctas. El sistema deberá rechazar el acceso cuando las credenciales sean inválidas.
RF3	**Cierre de sesión**	El sistema deberá permitir a los usuarios cerrar su sesión activa en la plataforma.	El sistema deberá finalizar la sesión del usuario autenticado. El sistema deberá impedir el acceso a funcionalidades protegidas después del cierre de sesión.
RF4	**Obtener usuario**	El sistema deberá permitir al usuario autenticado consultar su	El sistema deberá permitir el acceso

	autenticado	información personal.	únicamente a usuarios autenticados. El sistema deberá mostrar la información correspondiente al usuario autenticado. El sistema no deberá permitir el acceso a información de otros usuarios.
RF5	**Actualización de datos de usuario autenticado**	El sistema deberá permitir al usuario autenticado actualizar su información personal.	El sistema deberá permitir únicamente a usuarios autenticados realizar la actualización. El sistema deberá validar la información enviada. El sistema deberá actualizar los datos del usuario en la base de datos. El sistema deberá reflejar los cambios en futuras consultas

4. Casos de Uso

CU1 – Registro de usuario (RF1)

Actores:

- Usuario nuevo

Descripción:

Este caso de uso describe el proceso mediante el cual un usuario crea una cuenta en la plataforma enviando su información al sistema.

Precondiciones:

- El usuario no cuenta con una cuenta registrada.
- El usuario puede realizar solicitudes al sistema.

Flujo Principal:

1. El usuario accede a la vista de registro y envía una solicitud de registro con su información.

2. El sistema recibe la solicitud.
3. El sistema verifica que la información esté completa y cumpla con el formato esperado.
4. El sistema verifica que el correo, CURP y nombre de usuario no estén registrados.
5. El sistema cifra la contraseña.
6. El sistema registra al usuario en la base de datos.
7. El sistema confirma el registro exitoso y redirige al usuario a la vista de inicio de sesión.

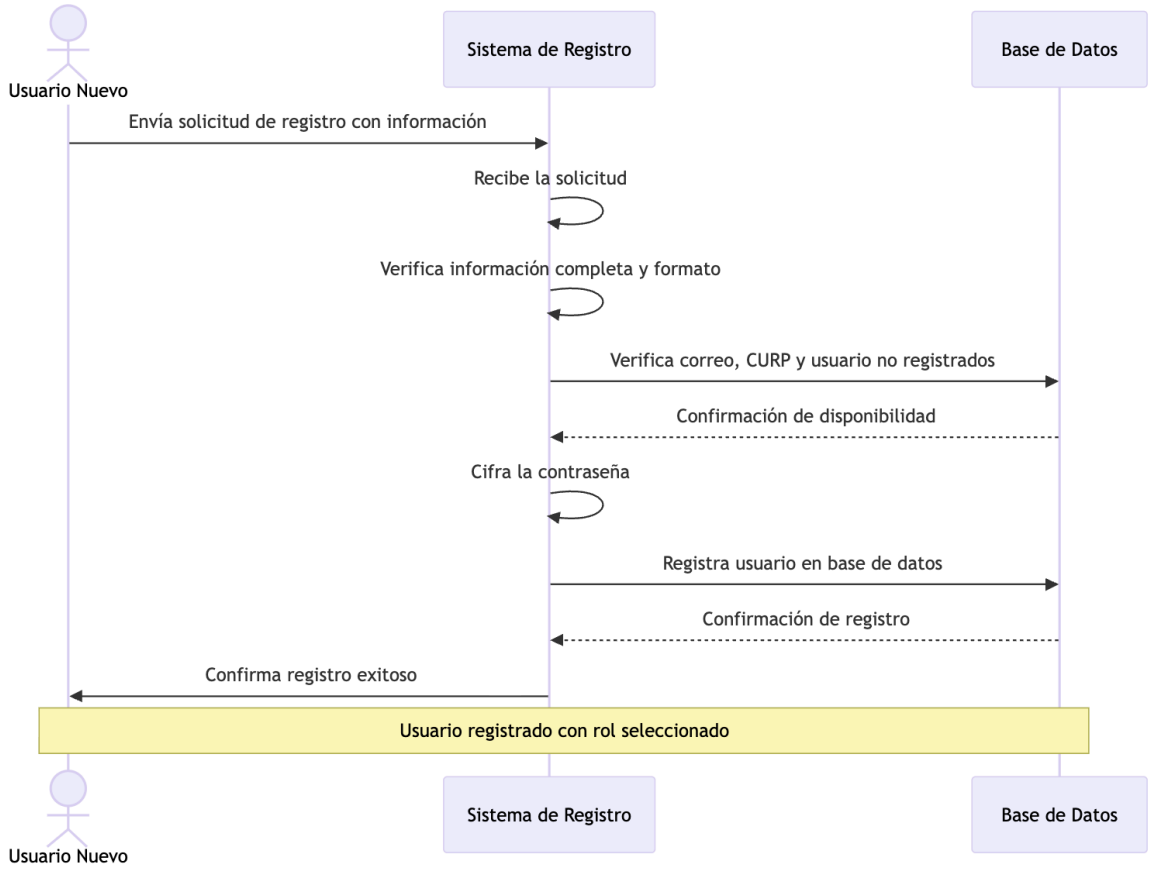
Flujos Alternativos:

- A1 – Información obligatoria faltante: Si el usuario omite uno o más campos obligatorios, el sistema muestra un mensaje de error indicando los campos pendientes. El sistema no registra al usuario.
- A2 – Correo electrónico o nombre de usuario ya registrado: Si el correo ingresado o el nombre de usuario ya existe en el sistema, el sistema muestra un mensaje indicando que el correo o nombre de usuario ya está en uso. El usuario deberá ingresar un correo, nombre de usuario diferente. El sistema no registra al usuario.
- A3 – Formato de entrada inválido: Si los datos ingresados no tiene un formato válido, el sistema muestra un mensaje de error. El sistema no registra al usuario.
- A4 – Contraseña no cumple requisitos: Si la contraseña no cumple con los requisitos mínimos de seguridad, el sistema muestra un mensaje indicando los criterios que deben cumplirse. El sistema no registra al usuario.

Postcondiciones:

- La cuenta del usuario queda registrada en el sistema con el rol seleccionado.

Diagrama:



CU2 – Inicio de sesión (RF2)

Actores:

- Usuario registrado

Descripción: Este caso de uso describe el proceso mediante el cual un usuario se autentica en el sistema.

Precondiciones:

- El usuario cuenta con una cuenta registrada y activa

Flujo Principal:

1. El usuario accede a la vista de login y envía una solicitud de autenticación con sus credenciales.

2. El sistema recibe la solicitud.

3. El sistema valida las credenciales.

4. El sistema confirma la autenticación y redirige al usuario a la vista principal (home).

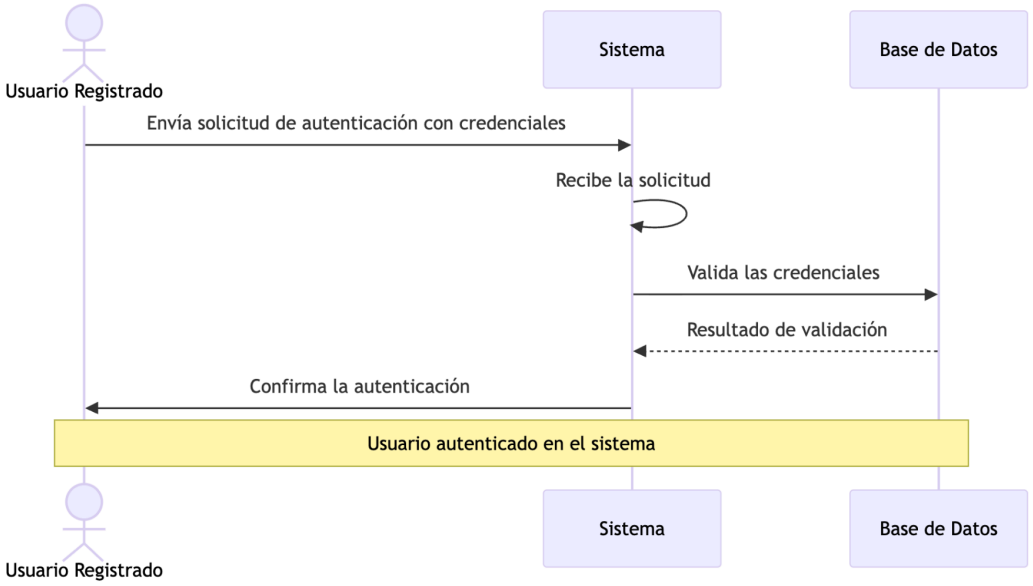
Flujos Alternativos:

- A1 – Credenciales incorrectas: Si el correo o la contraseña ingresados no coinciden con ningún registro, el sistema muestra un mensaje de error genérico sin especificar cuál campo es incorrecto. El sistema rechaza la solicitud de autenticación.

Postcondiciones:

- El usuario queda autenticado en el sistema.

Diagrama:



CU3 – Obtener usuario autenticado (RF4)

Actores:

- Usuario autenticado

Descripción: Este caso de uso describe cómo un usuario autenticado solicita su información personal al sistema.

Precondiciones:

- El usuario está autenticado.

Flujo Principal:

1. El usuario accede a la vista de perfil y envía una solicitud para obtener su información.
2. El sistema valida la autenticación del usuario.
3. El sistema recupera la información del usuario.
4. El sistema devuelve la información solicitada y se la muestra al usuario en la vista de perfil.

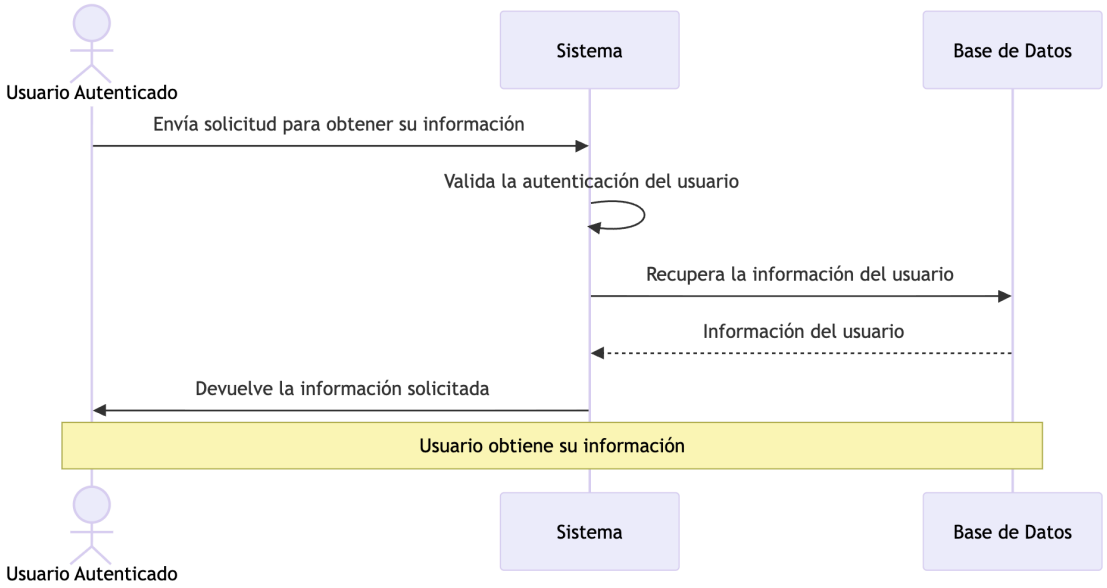
Flujos Alternativos:

- A1 - Usuario no autenticado: Se manda un mensaje de error explicando el problema y la acción de solución (autenticarse). El sistema rechaza la solicitud.

Postcondiciones:

- El usuario obtiene su información.

Diagrama:



CU4 – Actualización de usuario (RF5)

Actores:

- Usuario autenticado

Descripción: Este caso de uso describe el proceso mediante el cual un usuario modifica su información personal dentro del sistema.

Precondiciones:

- El usuario está autenticado.

Flujo Principal:

1. El usuario accede a la vista de perfil y envía una solicitud para editar su perfil.
2. El usuario edita su información personal.
3. El usuario envía la solicitud de actualización.
4. El sistema valida la autenticación del usuario.
5. El sistema valida los datos enviados.
6. El sistema actualiza la información en la base de datos.
7. El sistema confirma la actualización.

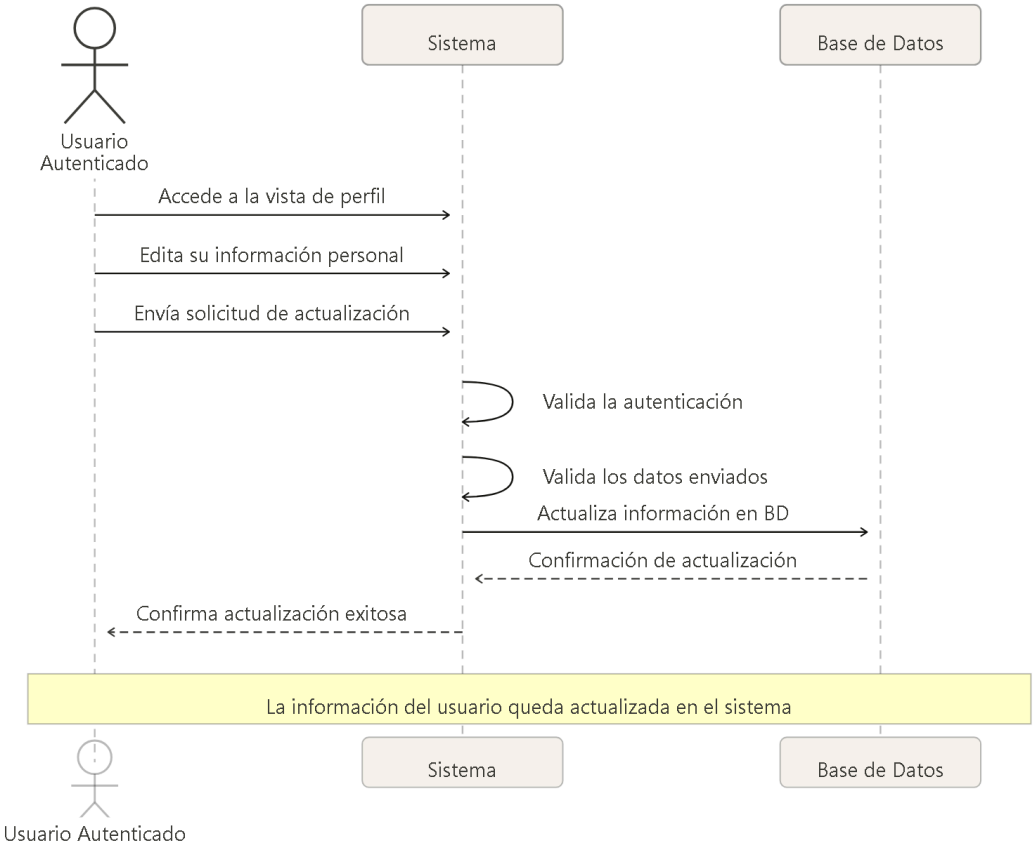
Flujos Alternativos:

- A1 - Usuario no autenticado: Se manda un mensaje de error explicando el problema y la acción de solución (autenticarse). El sistema rechaza la solicitud.
- A2 – Datos inválidos: El sistema muestra un mensaje de error indicando los campos incorrectos.

Postcondiciones:

- La información del usuario queda actualizada en el sistema.

Diagrama:



CU5 – Cierre de sesión (RF3)

Actores:

- Usuario autenticado

Descripción: Este caso de uso describe el proceso mediante el cual un usuario finaliza su sesión en el sistema.

Precondiciones:

- El usuario está autenticado.

Flujo Principal:

1. El usuario selecciona la opción de cerrar sesión desde la barra de navegación y envía una solicitud para cerrar sesión.

2. El sistema recibe la solicitud.

3. El frontend finaliza la sesión del usuario e invalida el token de sessionStorage y redirige al usuario a la vista de login.

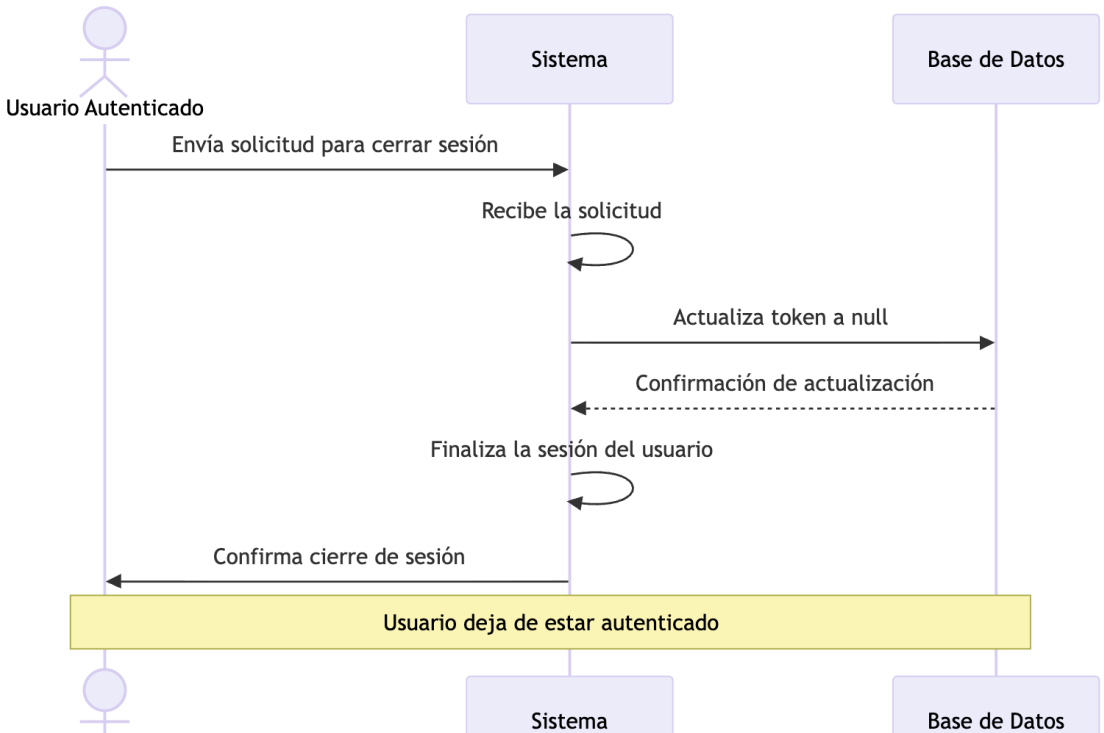
Flujos Alternativos:

- A1 - Usuario no autenticado: Se manda un mensaje de error explicando el problema y la acción de solución (autenticarse). El sistema rechaza la solicitud.

Postcondiciones:

- El usuario deja de estar autenticado.

Diagrama:



5. Requerimientos No Funcionales

Tipo		Descripción	Criterios de aceptación	RF
Seguridad	**RNF1**	Almacenamiento seguro de contraseñas. Las contraseñas deberán almacenarse mediante un algoritmo de cifrado seguro.	Las contraseñas de los usuarios deberán almacenarse de forma irreversible mediante un algoritmo de cifrado unidireccional. En ningún caso se almacenarán en texto plano.	RF1
	RNF2	Ninguna interfaz o funcionalidad del sistema estará disponible para usuarios no autenticados.	El sistema deberá requerir autenticación para acceder a endpoints protegidos. Las solicitudes sin autenticación válida deberán ser rechazadas. El sistema no deberá permitir acceso a información sin autenticación.	RF2
Rendimiento	**RNF3**	Capacidad de soportar usuarios concurrentes.	El sistema deberá mantener un tiempo de respuesta menor a 3 segundos con hasta 5,000 usuarios concurrentes.	
Disponibilidad	**RNF4**	Disponibilidad garantizada según los acuerdos nivel de servicio	El sistema deberá garantizar una disponibilidad del 98%.	
Accesibilidad y Usabilidad	**RNF5**	Mostrar errores claros y comprensibles	El sistema debe explicar al usuario que salió mal y qué acción debe tomar para corregirlo. Se debe tener mensajes para errores de formato en registro,	

			errores de inicio de sesión, errores de red, etc.	
Privacidad	**RNF6**	No mostrar información personal a terceros	La información mostrada a terceros como zip no se pueden usar para identificar de manera única La información personal como nombre y correo electrónico se almacenarán con estricta seguridad y no será disponible para nadie fuera de los administradores del sistema y el propio usuario	RF4, RF5
Mantenibilidad	**RFN7**	El código debe ser modular con una clara separación de capas	Utiliza MVC adaptado a servicios REST	
	RFN8	Pruebas	Incluye pruebas en postman para validar el funcionamiento	

6. Tecnologías Utilizadas

a) Lenguaje de programación en back-end: **Kotlin**

Kotlin fue el lenguaje de programación escogido para el back-end durante los laboratorios de la materia, Kotlin nos brinda compatibilidad con java y podemos usar el ecosistema de bibliotecas de Java, además de tener una sintaxis más clara que otros lenguajes de programación de JVM lo cual nos permite hacer mejores aplicaciones, robustas, seguras y fáciles de mantener; es compatible con frameworks modernos y podemos desarrollar APIs REST.

b) Framework en back-end: **Spring Boot**

Este es el framework que utilizaremos para el back-end, nos ayuda a la creación de aplicaciones REST que sean modulares y podemos tener una arquitectura en capas, además de tener soporte nativo y cuenta con una amplia documentación.

c) Base de datos: **Postgres**

Elegimos esta base de datos dado que es robusta y que la gran parte del equipo ya tenía familiaridad con la misma por lo que eso nos facilita el trabajar todos con ella, además de ser estable, nos brinda confiabilidad, ya que maneja los datos de manera segura.

d) <u>Hibernate:</u>

Permite la interacción entre el sistema y la base de datos de manera más eficiente, facilita la conversión de objetos del lenguaje de programación a registros en la base de datos.

e) **<u>Postman:</u>**

Herramienta de prueba para los endpoints, envía solicitudes HTTP y verifica las respuestas del sistema.

f) <u>Git:</u>

Sistema de control de versiones la cual nos ayuda a tener un seguimiento de los cambios en el sistema y mantener las versiones estables.

g) Frontend: **<u>NextJS</u>**

Se utilizó Next.js como framework para el desarrollo del frontend debido a su capacidad para construir aplicaciones modernas basadas en **<u>React</u>**, así como su soporte para renderizado eficiente y organización estructurada del proyecto.

h) Framework en front-end: **<u>TailwindCSS</u>**

Se utilizó Tailwind CSS como framework de estilos para agilizar el desarrollo de interfaces consistentes sin necesidad de escribir grandes cantidades de CSS personalizado.

- Además, se utilizó **<u>DaisyUI</u>** como complemento de Tailwind CSS para proporcionar componentes visuales predefinidos

i) Lenguaje de programación en front-end: **<u>TypeScript</u>**

Se utilizó TypeScript como lenguaje de programación en el front-end, el cual extiende JavaScript incorporando tipado estático. Esto permite mejorar la calidad del código y facilitar el mantenimiento del sistema.

El uso de TypeScript contribuye a una mayor robustez en la aplicación, especialmente en la interacción con la API del back-end, al definir estructuras claras para los datos enviados y recibidos.

j) Mecanismo de comunicación con back-end: **Axios**

Se utilizó Axios como biblioteca para la comunicación con el back-end mediante HTTP. Permite centralizar la configuración de las peticiones, manejar respuestas de forma sencilla y aplicar interceptores para la autenticación, evitando la repetición de código en cada llamada.

7. Arquitectura General

Patrón utilizado: **MVC adaptado a servicios REST**

Este patrón de diseño está implementado mediante una estructura en capas (Modelo, Vista, Controlador) lo cual nos permite una mejor separación de responsabilidades, facilita el mantenimiento y el desarrollo colaborativo. Pero a partir de la Iteración 2, el sistema funciona como un conjunto de dos componentes independientes que solo se comunican entre sí.

Arquitectura en capas:

a) **Controller:**

Es el responsable de recibir las peticiones HTTP del cliente, valida los datos, usa los servicios correspondientes y devuelve la respuesta en formato JSON, este es el intermediario entre el usuario y el interior de la aplicación.

b) Service:

Aquí se encuentra el comportamiento de nuestra aplicación, procesa la información mandada por el controller, aplica las reglas de negocio y gestiona procesos.

c) **Repository:**

Es la capa encargada del manejo de la base de datos, facilita la modificación futura del sistema de almacenamiento si fuera necesario, realiza consultas, guarda datos, recupera y actualiza datos.

Componentes del sistema:

a) **Frontend:**

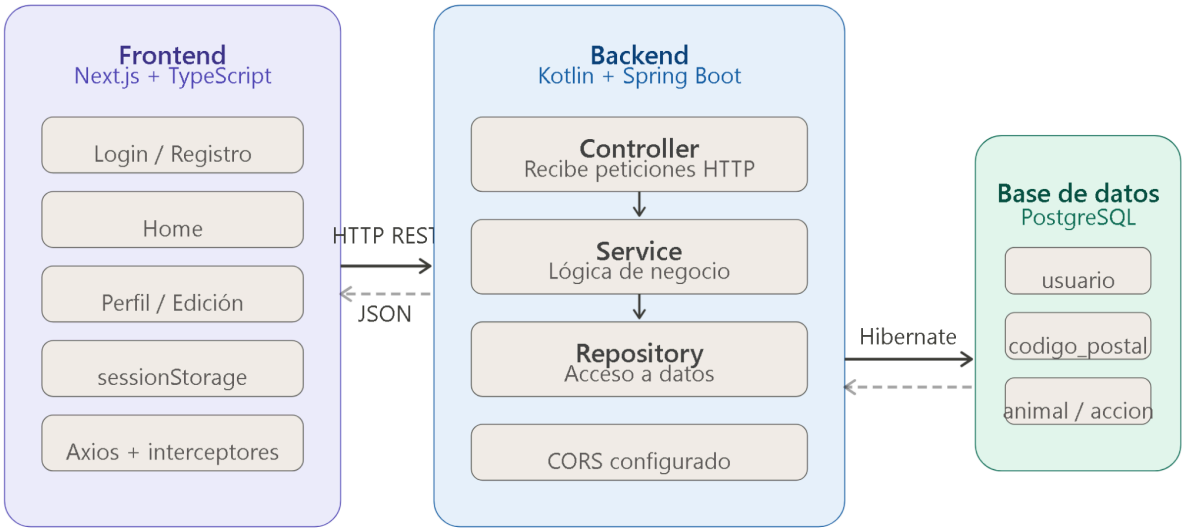
Es la interfaz destinada para el usuario y esta fue desarrollada con Next.js que utiliza la API del backend por medio de peticiones HTTP, el cual puede mostrar las vistas al usuario, gestionar la sesión activa en el navegador y proteger las rutas que requieran autenticación, de esta forma coordina la interacción del usuario con el backend.

b) **Backend:**

API REST se organiza en tres capas, una de ellas es el Controller el cual recibe las peticiones HTTP del frontend, validando los datos y devolviendolos como respuestas en formato JSON, además Service genera tokens, valida las credenciales y aplica las reglas del sistema donde Repository es nuestra capa de acceso a los datos(como la consulta, la actualización, registros), que se guardan en la base de datos mediante Hibernate.

c) **Comunicación:**

El frontend realiza peticiones HTTP al backend utilizando Axios, en donde cada petición que requiera alguna autentificación viene con un token en el encabezado Authorization: Bearer <token>. Además el backend tiene CORS configurado para poder aceptar peticiones del frontend y obtener una respuesta en formato JSON.



8. Flujo de Autenticación

El sistema implementa un flujo de autenticación basado en tokens para gestionar la sesión de los usuarios.

1. El usuario ingresa sus credenciales en la interfaz de login.
2. El front-end envía una solicitud al back-end.
3. El back-end valida las credenciales.
4. Si son correctas, el back-end genera un token de sesión.
5. El front-end almacena el token en sessionStorage.
6. En cada solicitud posterior, el front-end envía el token en el header Authorization.
7. El back-end valida el token antes de permitir acceso a endpoints protegidos.

En caso de que el token no exista o sea inválido:

- El back-end rechaza la solicitud con un error de autenticación.
- El front-end redirige al usuario a la vista de login.

Así, este mecanismo garantiza que solo usuarios autenticados puedan acceder a funcionalidades protegidas del sistema.

9. Modelo Conceptual de Datos

Este sistema se basa en la gestión de **usuarios** los cuales podrán acceder o bien interactuar con la aplicación, esta entidad cuenta con distintos atributos que son clave para la identificación y la gestión de cada usuario dentro de la aplicación.

Atributos principales:

a) **IDUsuario:** Este atributo de tipo UUID (Universally unique identifier) es el identificador único de un usuario dentro del sistema, es la llave de este atributo dentro de la base de datos.
b) **curp:** Este atributo de tipo VARCHAR(18), es la clave única de registro de población la cual nos servirá para la autenticación del usuario y que solo esté asociada a una cuenta.
c) **username:** Este atributo de tipo VARCHAR(50), es el nombre con el que se identificará el usuario en su perfil y al interactuar con otros.
d) **rol:** Este atributo de tipo rol_enum con valores ADOPTANTE, CUIDADOR y ADMINISTRADOR, es el tipo de un usuario que está utilizando la aplicación (Cuidador o interesado).
e) **foto_perfil:** Atributo de tipo VARCHAR(255) (URL), guarda la foto del usuario, esto permitirá que los usuarios conozcan al cuidador o interesado.
f) **nombres:** Atributo de tipo VARCHAR(100), contiene el nombre real del usuario registrado, lo cual es información mínima necesaria para la autenticación.
g) **apellido_paterno:** Atributo de tipo VARCHAR(100), contiene el apellido paterno del usuario registrado, lo cual es información mínima necesaria para la autenticación.
h) **apellido_materno:** Atributo de tipo VARCHAR(100), contiene el apellido materno del usuario registrado, lo cual es información mínima necesaria para la autenticación.
i) **email:** Atributo de tipo VARCHAR(100), es la dirección de correo electrónico utilizada como identificador de inicio de sesión.
j) **codigo_postal:** Atributo de tipo VARCHAR(5), nos permite conocer en donde se encuentra el cuidador o interesado.
k) **password:** Atributo de tipo VARCHAR(255), guarda la contraseña del usuario la cual se cifrará, esto ayudará al usuario a acceder a la cuenta si es suya.

l) **token:** Atributo de tipo VARCHAR(255), sirve para autenticar al usuario después de iniciar sesión.
m) **fecha_registro:** Atributo de tipo TIMESTAMP. sirve para saber la fecha y hora en que se creó la cuenta.
n) **fecha_update:** Atributo de tipo TIMESTAMP. sirve para saber la fecha y hora de la última actualización del perfil.
o) fecha_eliminado_ Atributo de tipo TIMESTAMP, sirve para saber fecha y hora que se elimino un perfil.

Consideraciones de seguridad:

Como ya comentamos en la parte de la contraseña, se utiliza un código de cifrado unidireccional y se guardará en la base de datos ya cifrada en este caso se utiliza SHA-256, esto evitará que se pueda recuperar en texto plano de la base de datos y que tampoco sea recuperable a partir del valor almacenado.

10. Limitaciones actuales

Actualmente, el sistema cuenta con una interfaz de usuario funcional; sin embargo, su alcance sigue limitado exclusivamente a la gestión de usuarios. Sin embargo, aún no se cuenta con la funcionalidad de eliminar usuario, tampoco se ha implementado la lógica de distinción entre vistas de adoptante y cuidador.

No se han implementado aún funcionalidades relacionadas con la gestión de animales, interacción entre usuarios o procesos de adopción.

Aún no tenemos ni la lógica ni un servidor donde los usuarios puedan almacenar sus imágenes y, por tanto, en lugar de subir un archivo como foto de perfil el sistema está aceptando una ruta URL de la foto almacenada en algún lugar tercero.

El sistema no cuenta aún con despliegue en un entorno de producción, por lo que su uso se limita a un entorno local.